Mini: Project documentation
Stage 03

Karel Šafr

1 Your project: stage 03

In this exercise, you will complete the identifier scanner and use it to translate a whole Mini program. The parser, C++ generator and runtime are already in your project. We will follow one input from characters to tokens, a syntax tree and two printed lines, so you can explain how your small change enables the rest of the pipeline.

Use the start version for your own work. The table below shows what already works and what your change will add. Follow the exercise in order, then compare your results with the solution when it is provided.

Area	**Start**	**After completion**
Build and CLI	The C++ project builds; --help works.	Unchanged.
Scanning	Numbers, strings, comments and punctuation work; a name reaches TODO.	Whole names are classified as keywords or identifiers.
Program syntax	Supplied parser cannot be reached with a normal main program.	One parameterless main; literal/ grouping expressions, print and expression statements.
Later phases	AST, identity lowering, empty resolver, C++ emission and runtime are supplied.	The same infrastructure can process supported whole programs.
Not included	Variables, operators, nested blocks, branches, loops, calls and returns.	Still excluded; recognizing their tokens does not add their grammar.

How to use this document

Begin with ASSIGNMENT.txt in this folder. Run the starting example, read the task, and then use this guide to trace the relevant code. The file reference explains all 31 source and configuration files. When I provide the solution, its READ_ME.txt will help you compare your approach with the worked solution.

You have everything you need for this exercise in the folder you received: the compiler source, runtime, build configuration, examples and checks. Our compiler translates Mini to C++17 and then uses a C++ compiler to produce a native executable. Follow these separate steps when you explain how a Mini program runs.

2 Build the compiler and make a first input

Requirements: CMake 3.20 or newer, a C++17 compiler and its standard library. The commands below use a macOS shell and Unix Makefiles. Run them in a writable working copy of the selected package, in the directory containing CMakeLists.txt. The project does not download third-party C++ libraries.

```
cmake --version
c++ --version
cmake -S . -B build -G "Unix Makefiles" \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel 2
./build/minic --help
```

The supplied inputs/ directory contains starting_point.mini, acceptance.mini and edge cases. Save this additional manual demonstration as inputs/hello.mini:

```
fun main() {
  print 42;
  print "Mini";
}
```

```
./build/minic tokens inputs/hello.mini
./build/minic ast inputs/hello.mini
./build/minic run inputs/hello.mini
```

After completing this exercise, run prints 42 and Mini on separate lines and exits 0. In the untouched start, scanning fun reaches TODO WEEK03. Use tokens with inputs/starting_point.mini to check your setup before implementing the scanner. Each later exercise has its own starting-point example.

Run the supplied baseline and checkpoint checks

```
./build/minic tokens inputs/starting_point.mini
python3 CHECK_PROJECT.py --mode start --compiler ./build/minic
# After editing and rebuilding:
python3 CHECK_PROJECT.py --mode solution --compiler ./build/minic
```

The baseline works before editing. Python 3.8+ is needed only for the optional checker. PASS includes matched expected errors; start-mode PASS does not mean completion. Rebuild after C++ edits, using a separate build for each stage. Later examples abbreviate ./build/minic as minic; substitute your executable path. ASSIGNMENT.txt lists the bundled inputs; other manual examples may need to be saved separately.

3 Architecture: calls, data and two builds

Order / data	Responsible component
1. SourceText	source.cpp
2. tokens	lexer.cpp
3. parsed AST	parser.cpp
4. core AST	lower.cpp
5. resolved AST	resolver.cpp
6. C++ text	codegen.cpp
7. native program	HostCompiler

driver.cpp coordinates the path; build/run add the external C++ compiler.

main.cpp is the process boundary; cli.cpp validates arguments. SourceText::fromFile reads bytes before executeCommand drives the shared pipeline. A command selects how far to proceed; it does not select an independent parser.

The two compilations are different

- CMake builds our C++ implementation into the mini_compiler static library and the minic executable. configure_file generates mini_build_config.hpp in the build directory.
- minic cpp emits your Mini program as C++ text. minic build/run invokes HostCompiler to compile that generated translation unit with the header-only runtime. The resulting program is a different executable.
- The runtime executes Value operations and printing when that generated executable runs. It is not the lexer, parser or resolver, and it is not linked as a separate runtime library.

Current tree and pass contracts

Program.main_body holds the statements in the single main function. Resolution is an empty result type. We do not need name analysis yet because this version has no variables.

For this stage, lowering is an identity pass. It is visible in the driver so the same pipeline can grow without changing the CLI.

SourceText owns the source string. Tokens own lexeme/payload data and retain source spans. AST variants encode node kinds; unique_ptr owns recursive children. Moves transfer ownership between parser/lowering results. This compiler memory is distinct from scope tables and from values in a running Mini program.

4 Commands, output files and errors

Command	Last phase / observable result
tokens FILE	scan; prints the token sequence including EOF. A fragment may be enough.
ast FILE	parse; prints the parsed tree before lowering and resolution.
lower FILE	lower; prints the core tree before resolution.
check FILE	resolve; success is silent, status 0. No C++ is emitted or executed.
cpp FILE [-o OUT.cpp]	emitCpp; writes text to stdout or to the given file.
build FILE [-o OUT]	host C++ compilation; creates an executable but does not run it.
run FILE	host C++ compilation then execution; uses and cleans a temporary directory.
--help	CLI only; no input is read. For the available features, use the opening scope table.

Without -o, build chooses an executable beside the input (normally the stem, with the platform suffix). Choose an explicit output name when you want to keep the generated file. cpp/build reject overwriting the Mini input. run forwards the generated program output and its exit status; it does not leave a reusable executable.

```
./build/minic cpp inputs/hello.mini -o build/hello.cpp
./build/minic build inputs/hello.mini -o build/hello
./build/hello
```

Status / phase	Interpretation
0	Requested operation succeeded; check/build can legitimately print nothing.
1 / lex, parse, resolve, codegen, io, backend	Controlled compiler error; inspect its phase and location where provided.
1 / runtime	Generated program failed during an operation; e.g. incompatible operands.
2 / cli	Wrong command/arguments. No-argument minic is not the help command.
70 / internal	Unexpected std::exception. The stage08 lowering TODO deliberately uses this path.

check is not a dynamic type checker and cannot prove that execution will succeed. A parsed program can still fail resolution; a resolved program can still fail code generation at a TODO or fail later at runtime. Follow the earliest failing phase.

5 Your starting point: names, tokens and a small language

Your task is to read one complete name-shaped word, then decide whether it is reserved. Your project already contains source loading, token records, the keyword table, other scanning routines, a small parser, an AST formatter, C++ generation and the native build/run wrapper. We will use those parts to observe your change through the whole translation pipeline.

You can build minic before editing identifier(). A tokens request for 123 already works because it uses number(). A complete Mini program starts with fun main(), so scanning fun immediately

reaches the TODO. Run the number example first to check your setup, then use the hello example to see what changes after you complete the method.

Data at the boundaries

Contract	Meaning in this stage
SourceText / SourceSpan	Owned source bytes and filename; byte offset/length and one-based line/column.
scan(source)	A vector of Token records ending with EOF.
Token	kind, original lexeme, source span, optional number/string payload.
Program	A vector named main_body; general function declarations are not represented yet.
Resolution	An empty result type; no names need semantic binding in the supported grammar.

The completed parser accepts exactly one parameterless main with print statements or expression statements. An expression is a number, string, Boolean, nil or parenthesized expression. Parentheses are retained as GroupingExpression nodes. There is no arithmetic parser: even unary minus is absent. An expression statement evaluates its expression later without printing it.

Lowering is the identity operation because no supported construct requires rewriting. The resolver is intentionally empty. TokenLiteral and runtime Value are separate variants: the first stores decoded token data; the second represents values in the generated program. Runtime supports value formatting and printValue, but no arithmetic operations in this package.

The full keyword and punctuation inventory is broader than this parser. FOR or PLUS in a token dump does not prove that loops or addition can run. The shared --help describes final Mini; use this stage-specific contract when deciding which programs should succeed.

6 Implement identifier(): contract, algorithm and boundaries

Change only the body of Lexer::identifier() in src/lexer.cpp. scanToken() has already consumed the first ASCII letter or underscore before it calls this method. start_ still points to that first character; current_ points to the next unread byte. The method returns void: its observable results are an appended token and an updated cursor, not a returned string.

The steps you need to implement

- Inspect the next unread character with peek(). Continue while the supplied isAsciiAlphaNumeric predicate accepts it; this includes ASCII letters, digits and underscore.
- Consume only accepted continuation characters with advance(). Leave a delimiter, punctuation character or EOF for the surrounding scanner to handle.
- Take the complete slice starting at start_ with length current_ - start_. This includes the first character already consumed by the caller.
- Pass that complete slice to identifierKind(), then append exactly one token using addToken(). Keep its existing span and lexeme construction.

Cursor example: the source fragment for2;

```
At identifier entry: start_ = 0, current_ = 1
Unread continuation: o, r, 2
At identifier exit:  start_ = 0, current_ = 4
Consumed lexeme:    for2
Still unread:       ;
```

This is a hand trace of cursor positions, not CLI stdout. The interval [start_, current_) must contain exactly the whole name. Classifying after only the three letters for would incorrectly split for2 into a keyword and a number. Conversely, consuming the semicolon would remove the token that the parser needs to recognize the end of a statement.

At EOF, peek() supplies a nonmatching sentinel, so the loop stops without reading beyond the buffer. Lexer::run() adds EOF afterwards; identifier() must not add another EOF. The supplied addToken() computes both source span and spelling from the same interval. Keywords use exact, case-sensitive matching: print is reserved, while Print and print2 are identifiers.

Remove the placeholder throw as part of the implementation. Merely deleting it would lose every name token. Do not replace the keyword table, broaden names to Unicode, change number scanning, add grammar rules or implement a new runtime operation. Rebuild your actual compiler before evaluating the result.

7 Worked translation: from hello text to two output lines

Save the following one-line source as w03_hello.mini in your project folder, followed by a newline. This is a manual example you create; the required checkpoint inputs are already in inputs/. Run the commands from that project folder. minic means the executable you have rebuilt, for example ./build/minic; substitute your actual build path.

```
fun main() { print 42; print "Mini"; }
```

Before the edit, minic run w03_hello.mini exits 1 with the message fragment "lex: TODO WEEK03: implement identifier scanning" at 1:1. Program stdout is empty. The same lexical failure affects ast, check and cpp: every command begins by scanning the original Mini file. No parser or generated program has processed this input yet.

After completion and rebuild

```
minic tokens w03_hello.mini
minic ast w03_hello.mini
minic lower w03_hello.mini
minic check w03_hello.mini
minic cpp w03_hello.mini
minic run w03_hello.mini
```

The token stream now includes FUN, the identifier main, delimiters, PRINT and the two literal tokens. parse consumes the vector, not the text printed by a previous tokens command. The complete ast stdout is shown below; lower produces the same text because this stage performs no rewrite. Each successful command returns 0.

```
(program
  (fun main ()
    (print (number 42))
    (print (string "Mini"))
  )
)
```

check succeeds silently after the empty resolver. cpp instead emits C++ text containing temporary Value objects and calls to mini::printValue. That text is not program output and has not executed. For run, the supplied host wrapper compiles the generated C++ with the runtime header, then launches the resulting executable. Its exact stdout is:

```
42
Mini
```

The two print statements keep their order through parsing and generation. The generated mini_ main function performs them when called by the generated C++ int main. That entry point is separate from src/main.cpp, which starts minic. As you read the output, connect each printed line back to its token, AST node and generated print call. Your identifier method makes those input tokens available.

8 Check your scanner and explain what it enables

Start with a baseline that must not regress

Create w03_number.mini in your project folder containing 123 and a final newline. minic tokens w03_number.mini should print exactly the two lines below before and after your edit. This is a scanner input, not a complete Mini program; use tokens for this check. The bundled inputs/ starting_point.mini contains the same fragment.

```
NUMBER 1:1 "123" = 123
EOF 2:1 ""
```

Check the rule, not only the large demonstration

For the lexical fragment for for2 _x3, the completed scanner must produce FOR, IDENTIFIER, IDENTIFIER and EOF. Inspect spelling and positions as well as kinds. Also try a final name without a trailing newline, a single underscore, a digit after the first character and a capitalized word. A following semicolon must remain available to the main scanner. These cases distinguish a whole-name algorithm from an implementation that only happens to recognize fun and print.

An intentional negative result

```
fun main() { print 1 + 2; }
```

Save that line as w03_operator.mini in your project folder. Before your edit, it reaches the identifier TODO. After your edit, tokens should succeed, while minic check w03_operator.mini should exit 1 during parse with "expected ';' after print value". The parser has read literal 1 and then finds PLUS where it requires a semicolon. Keep this rejection: we have not introduced arithmetic expressions in this exercise.

- Explain why for2 is one token and why its delimiter remains unread.
- Show unchanged number scanning, an EOF-terminated token vector, the hello AST, a silent successful check and exact hello output.
- Use the phase label to distinguish a lexer error, unsupported syntax and a host build failure. Explain which part of the pipeline produced each result.

Keep your completed scanner and your examples. In Stage 04, we will implement declaration IDs, initialization checks and block traversal. I will provide the parser, AST and C++ generation for

variables and blocks. For now, be ready to explain why scanning a keyword is necessary but does not itself implement its grammar.

9 File reference: responsibilities

Use this table to locate the code in your project folder. The two counts give each file's total length in start and solution, including comments and blank lines. They are not line numbers to edit. The worked solution adds 5 nonblank code lines, excluding comments and the unchanged skeleton. Your implementation may use a different number of lines; follow the task and explain how your code produces the required behavior.

File	Total lines: start / solution	Responsibility and boundary
.gitignore	7 / 7	Version-control exclusions for generated build/output files. It is configuration, not a compiler phase.
CMakeLists.txt	57 / 57	Requires CMake 3.20 and C++17; configures mini_ build_config.hpp, builds mini_compiler and links minic. Run the bundled checkpoint checks separately.
runtime/mini_runtime.hpp	46 / 46	Header-only Value representation, printing and the operations available in this stage. Included by generated programs, not a replacement for the frontend.
src/ast.cpp	77 / 77	formatAst: readable structural dump shared by ast and lower. This is a view of the tree, not an evaluator or stable serialization format.
src/ast.hpp	67 / 67	Owns the stage-specific Program, Expression and Statement data structures. Tagged variants distinguish node kinds; unique_ptr owns recursive children.
src/build_config.hpp.in	15 / 15	CMake template for compiler path/family, runtime include directory and executable suffix. Its generated header belongs to the build directory.
src/cli.cpp	129 / 129	Validates command syntax and -o usage; formats common help. The final-language help paragraph does not promise all features in an early stage.
src/cli.hpp	37 / 37	CommandKind and CommandLine: command, input path and optional output path. Declares parse CommandLine and helpText.
src/codegen.cpp	159 / 159	Traverses the current tree, emits values/statements, and creates the host main wrapper. See the stage-specific sections for supported nodes and the exercise.
src/codegen.hpp	19 / 19	emitCpp(const Program&, const Resolution&) returns one C++17 translation unit. It consumes the resolved tree; it does not launch a compiler.
src/diagnostics.cpp	69 / 69	Maps diagnostic phases to labels and formats source-name/line/column when available. Supplies consistent compiler-side error messages.
src/diagnostics.hpp	49 / 49	Phase and CompileError contracts, optional source location and diagnostic formatting interface. Compile Error is distinct from RuntimeError.
src/driver.cpp	200 / 200	Runs phases in order and stops at the requested view. Writes C++, manages temporary directories and invokes HostCompiler for build/run.

File	Total lines: start / solution	Responsibility and boundary
src/driver.hpp	20 / 20	executeCommand(const CommandLine&, const SourceText&, ostream&) is the shared orchestration entry point used by the executable.
src/host_compiler.cpp	281 / 281	Selects and invokes the external C++ toolchain, quotes arguments, captures compilation logs and normalizes process status. Checks fresh executable output.
src/host_compiler.hpp	44 / 44	HostCompilerConfig and HostCompiler: configured toolchain, compile(source, executable, scratch), and run(executable).
src/lexer.cpp	243 / 246	Reads characters and creates tokens with spans and literal payloads, skipping whitespace/comments. Keyword recognition and grammar acceptance are separate.
src/lexer.hpp	19 / 19	scan(const SourceText&) returns vector<Token>. The public function hides the scanner cursor and token-building helpers.
src/lower.cpp	14 / 14	Provides the explicit lowering stage. Identity before stage08; from stage08 rewrites parsed for into core blocks and while after the exercise is completed.
src/lower.hpp	17 / 17	lower(Program) returns Program. Taking the tree by value supports moving its owned children instead of copying them.
src/main.cpp	41 / 41	Process entry: collects argv, parses CLI, loads Source Text and calls driver. Catches compiler errors and unexpected C++ exceptions at the boundary.
src/number_format.cpp	43 / 43	Implements locale-independent numeric formatting used by diagnostics/dumps and generated code; avoids decimal-comma differences.
src/number_format.hpp	17 / 17	Declares formatNumberForDisplay and formatNumber ForCpp. A human-readable number and a valid C++ double literal serve different purposes.
src/parser.cpp	163 / 163	Recursive-descent parser for the current stage. Constructs AST nodes and reports the first syntax error; does not execute expressions or resolve names.
src/parser.hpp	20 / 20	parse(const SourceText&, const vector<Token>&) returns Program. Source text supports diagnostics; tokens supply the grammar input.
src/resolver.cpp	14 / 14	Name-analysis pass: no-op in stage03, local/scope binding from stage04, outward lookup after stage05 and function resolution from stage09.
src/resolver.hpp	19 / 19	resolve(const SourceText&, Program&) returns Resolution and may annotate AST names. The exact result and lookup capabilities grow by stage.
src/source.cpp	77 / 77	Loads the input file, indexes line starts and maps offsets to source positions. Shared by lexical, syntax and semantic diagnostics.
src/source.hpp	44 / 44	SourceText owns source name, contents and line starts; SourceSpan stores offset, length, line and column. Declares file loading and span lookup.
src/token.cpp	119 / 119	formatTokens: prints token kinds, locations, lexemes and payloads. EndOfFile is explicit; token names describe scanning, not full language support.

File	Total lines: start / solution	Responsibility and boundary
src/token.hpp	66 / 66	TokenKind, TokenLiteral and Token. Each token owns lexeme text, a source span and an optional double/ string literal payload.

Use CHECK_PROJECT.py, checks/CASES.json and inputs/ for the checks described in ASSIGNMENT.txt. These support files are separate from the compiler sources listed here. A worked explanation accompanies the solution.

10 Working on your project

Keep the build and runtime together

Keep runtime/ beside src/. If you move the project, configure and build it again in its new location; the build records paths to the compiler and runtime. HostCompiler uses MINI_CXX if set, then CXX, then the compiler chosen by CMake. These overrides take one executable name or path, without extra flags.

Run the project in your IDE

In CLion, open the folder containing CMakeLists.txt and build the minic target. Set the program arguments to tokens inputs/starting_point.mini. Set the working directory to the project folder. Your IDE may use its own build directory; use the minic executable from that directory when following the command-line examples.

What you see	What to do
Cannot open input	Check the working directory and filename, including the .mini extension.
The old TODO still appears	Rebuild. Check that you are running minic from the copy you edited.
A backend error	Read the compiler log in the error. Check your C++ toolchain and the runtime path.
An unsupported feature	Check the opening scope table and the examples for this stage.
CTest finds no tests	Use CHECK_PROJECT.py or the manual commands in ASSIGNMENT.txt.

Before you finish

Your change in this exercise is in src/lexer.cpp. Rebuild, run the acceptance example and complete the edge cases in ASSIGNMENT.txt. The solution mode of CHECK_PROJECT.py checks your edited start as well as the worked solution.

- Show an example that works before your change, then explain the new behavior after your change.
- Point to the function you changed. Explain the order of its steps and why the edge cases matter.
- For a rejected example, identify the failing phase and explain why that error is expected.

Keep your completed work

Save this completed project separately. I will provide a new starting project for our next exercise. It will include the supporting changes needed for that topic. Open and build that new project as a whole; copying one file between versions can mix incompatible parts.